

YOLOv4

This is PyTorch implementation of YOLOv4 which is based on [ultralytics/yolov3](#).

- [\[original Darknet implementation of YOLOv4\]](#)
- [\[ultralytics/yolov5 based PyTorch implementation of YOLOv4\]](#).

development log

► Expand

- 2021-10-31 - support [RS loss](#), [aLRP loss](#), [AP loss](#).
- 2021-10-30 - support [alpha IoU](#).
- 2021-10-20 - design resolution calibration methods.
- 2021-10-15 - support joint detection, instance segmentation, and semantic segmentation. [seg-yolo](#)
- 2021-10-13 - design ratio yolo.
- 2021-09-22 - pytorch 1.9 compatibility.
- 2021-09-21 - support [DIM](#).
- 2021-09-16 - support [Dynamic Head](#).
- 2021-08-28 - design domain adaptive training.
- 2021-08-22 - design re-balance models.
- 2021-08-21 - support [simOTA](#).
- 2021-08-14 - design approximation-based methods.
- 2021-07-27 - design new decoders.
- 2021-07-22 - support 1) decoupled head, 2) anchor-free, and 3) multi positives in [yolox](#).
- 2021-07-10 - design distribution-based implicit modeling.
- 2021-07-06 - support outlooker attention. [volo](#)
- 2021-07-06 - design self ensemble training method.
- 2021-06-23 - design cross multi-stage correlation module.
- 2021-06-18 - design cross stage cross correlation module.
- 2021-06-17 - support cross correlation module. [ccn](#)
- 2021-06-17 - support attention modules. [cbam](#) [saan](#)
- 2021-04-20 - support swin transformer. [swin](#)
- 2021-03-16 - design new stem layers.
- 2021-03-13 - design implicit modeling. [nnmf lc](#)
- 2021-01-26 - support vision transformer. [tr](#)
- 2021-01-26 - design mask objectness.
- 2021-01-25 - design rotate augmentation.
- 2021-01-23 - design collage augmentation.
- 2021-01-22 - support [VoVNet](#), [VoVNet2](#).
- 2021-01-22 - support [EIoU](#).
- 2021-01-19 - support instance segmentation. [mask-yolo](#)
- 2021-01-17 - support anchor-free-based methods. [center-yolo](#)
- 2021-01-14 - support joint detection and classification. [classify-yolo](#)

- 2020-01-02 - design new [PRN](#) and [CSP](#)-based models.
- 2020-12-22 - support transfer learning.
- 2020-12-18 - support non-local series self-attention blocks. [gc dn1](#)
- 2020-12-16 - support down-sampling blocks in cspnet paper. [down-c down-d](#)
- 2020-12-03 - support imitation learning.
- 2020-12-02 - support [squeeze and excitation](#).
- 2020-11-26 - support multi-class multi-anchor joint detection and embedding.
- 2020-11-25 - support [joint detection and embedding](#). [track-yolo](#)
- 2020-11-23 - support teacher-student learning.
- 2020-11-17 - pytorch 1.7 compatibility.
- 2020-11-06 - support inference with initial weights.
- 2020-10-21 - fully supported by darknet.
- 2020-09-18 - design fine-tune methods.
- 2020-08-29 - support [deformable kernel](#).
- 2020-08-25 - pytorch 1.6 compatibility.
- 2020-08-24 - support channel last training/testing.
- 2020-08-16 - design CSPPRN.
- 2020-08-15 - design deeper model. [csp-p6-mish](#)
- 2020-08-11 - support [HarDNet](#). [hard39-pacsp hard68-pacsp hard85-pacsp](#)
- 2020-08-10 - add DDP training.
- 2020-08-06 - support [DCN](#), [DCNv2](#). [yolov4-dcn](#)
- 2020-08-01 - add pytorch hub.
- 2020-07-31 - support [ResNet](#), [ResNeXt](#), [CSPResNet](#), [CSPResNeXt](#). [r50-pacsp x50-pacsp cspr50-pacsp cspx50-pacsp](#)
- 2020-07-28 - support [SAM](#). [yolov4-pacsp-sam](#)
- 2020-07-24 - update api.
- 2020-07-23 - support CUDA accelerated Mish activation function.
- 2020-07-19 - support and training tiny YOLOv4. [yolov4-tiny](#)
- 2020-07-15 - design and training conditional YOLOv4. [yolov4-pacsp-conditional](#)
- 2020-07-13 - support [MixUp](#) data augmentation.
- 2020-07-03 - design new stem layers.
- 2020-06-16 - support floating16 of GPU inference.
- 2020-06-14 - convert .pt to .weights for darknet fine-tuning.
- 2020-06-13 - update multi-scale training strategy.
- 2020-06-12 - design scaled YOLOv4 follow [ultralytics](#). [yolov4-pacsp-s yolov4-pacsp-m yolov4-pacsp-l yolov4-pacsp-x](#)
- 2020-06-07 - design [scaling methods](#) for CSP-based models. [yolov4-pacsp-25 yolov4-pacsp-75](#)
- 2020-06-03 - update COCO2014 to COCO2017.
- 2020-05-30 - update FPN neck to CSPFPN. [yolov4-yocsp yolov4-yocsp-mish](#)
- 2020-05-24 - update neck of YOLOv4 to CSPPAN. [yolov4-pacsp yolov4-pacsp-mish](#)
- 2020-05-15 - training YOLOv4 with Mish activation function. [yolov4-yospp-mish yolov4-paspp-mish](#)
- 2020-05-08 - design and training YOLOv4 with [FPN](#) neck. [yolov4-yospp](#)
- 2020-05-01 - training YOLOv4 with Leaky activation function using PyTorch. [yolov4-paspp PAN](#)

Pretrained Models & Comparison

Model	Test Size	AP ^{test}	AP ₅₀ ^{test}	AP ₇₅ ^{test}	AP _S ^{test}	AP _M ^{test}	AP _L ^{test}	cfg	weights
YOLOv4	640	50.0%	68.4%	54.7%	30.5%	54.3%	63.3%	cfg	weights
YOLOv4 _{pacsp-s}	640	39.0%	57.8%	42.4%	20.6%	42.6%	50.0%	cfg	weights
YOLOv4 _{pacsp}	640	49.8%	68.4%	54.3%	30.1%	54.0%	63.4%	cfg	weights
YOLOv4 _{pacsp-x}	640	52.2%	70.5%	56.8%	32.7%	56.3%	65.9%	cfg	weights
YOLOv4 _{pacsp-s-mish}	640	40.8%	59.5%	44.3%	22.4%	44.6%	51.8%	cfg	weights
YOLOv4 _{pacsp-mish}	640	50.9%	69.4%	55.5%	31.2%	55.0%	64.7%	cfg	weights
YOLOv4 _{pacsp-x-mish}	640	52.8%	71.1%	57.5%	33.6%	56.9%	66.6%	cfg	weights
Model	Test Size	AP ^{val}	AP ₅₀ ^{val}	AP ₇₅ ^{val}	AP _S ^{val}	AP _M ^{val}	AP _L ^{val}	cfg	weights
YOLOv4	640	49.7%	68.2%	54.3%	32.9%	54.8%	63.7%	cfg	weights
YOLOv4 _{pacsp-s}	640	38.9%	57.7%	42.2%	21.9%	43.3%	51.9%	cfg	weights
YOLOv4 _{pacsp}	640	49.4%	68.1%	53.8%	32.7%	54.2%	64.0%	cfg	weights
YOLOv4 _{pacsp-x}	640	51.6%	70.1%	56.2%	35.3%	56.4%	66.9%	cfg	weights
YOLOv4 _{pacsp-s-mish}	640	40.7%	59.5%	44.2%	25.3%	45.1%	53.4%	cfg	weights
YOLOv4 _{pacsp-mish}	640	50.8%	69.4%	55.4%	34.3%	55.5%	65.7%	cfg	weights
YOLOv4 _{pacsp-x-mish}	640	52.6%	71.0%	57.2%	36.4%	57.3%	67.6%	cfg	weights
► archive									
Model	Test Size	AP ^{val}	AP ₅₀ ^{val}	AP ₇₅ ^{val}	AP _S ^{val}	AP _M ^{val}	AP _L ^{val}	cfg	weights
YOLOv4	640	48.4%	67.1%	52.9%	31.7%	53.8%	62.0%	cfg	weights

Model	Test Size	AP ^{val}	AP ₅₀ ^{val}	AP ₇₅ ^{val}	AP _S ^{val}	AP _M ^{val}	AP _L ^{val}	cfg	weights
YOLOv4 _{pacsp-s}	640	37.0%	55.7%	40.0%	20.2%	41.6%	48.4%	cfg	weights
YOLOv4 _{pacsp}	640	47.7%	66.4%	52.0%	32.3%	53.0%	61.7%	cfg	weights
YOLOv4 _{pacsp-x}	640	50.0%	68.3%	54.5%	33.9%	55.4%	63.7%	cfg	weights
YOLOv4 _{pacsp-s-mish}	640	38.8%	57.8%	42.0%	21.6%	43.7%	51.1%	cfg	weights
YOLOv4 _{pacsp-mish}	640	48.8%	67.2%	53.4%	31.5%	54.4%	62.2%	cfg	weights
YOLOv4 _{pacsp-x-mish}	640	51.2%	69.4%	55.9%	35.0%	56.5%	65.0%	cfg	weights
Model	Test Size	AP ^{val}	AP ₅₀ ^{val}	AP ₇₅ ^{val}	AP _S ^{val}	AP _M ^{val}	AP _L ^{val}	cfg	weights
YOLOv4	672	47.7%	66.7%	52.1%	30.5%	52.6%	61.4%	cfg	weights
YOLOv4 _{pacsp-s}	672	36.6%	55.5%	39.6%	21.2%	41.1%	47.0%	cfg	weights
YOLOv4 _{pacsp}	672	47.2%	66.2%	51.6%	30.4%	52.3%	60.8%	cfg	weights
YOLOv4 _{pacsp-x}	672	49.3%	68.1%	53.6%	31.8%	54.5%	63.6%	cfg	weights
YOLOv4 _{pacsp-s-mish}	672	38.6%	57.7%	41.8%	22.3%	43.5%	49.3%	cfg	weights
(+BoF)	640	39.9%	59.1%	43.1%	24.4%	45.2%	51.4%		weights
YOLOv4 _{pacsp-mish}	672	48.1%	66.9%	52.3%	30.8%	53.4%	61.7%	cfg	weights
(+BoF)	640	49.3%	68.2%	53.8%	31.9%	54.9%	62.8%		weights
YOLOv4 _{pacsp-x-mish}	672	50.0%	68.5%	54.4%	32.9%	54.9%	64.0%	cfg	weights
(+BoF)	640	51.0%	69.7%	55.5%	33.3%	56.2%	65.5%		weights

Requirements

docker (recommanded):

```
# create the docker container, you can change the share memory size if you
have more.
nvidia-docker run --name yolov4 -it -v your_coco_path/./coco/ -v
your_code_path/./yolo --shm-size=64g nvcr.io/nvidia/pytorch:20.11-py3

# apt install required packages
apt update
apt install -y zip http screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop

# install mish-cuda if you want to use mish activation
# https://github.com/thomasbrandon/mish-cuda
# https://github.com/JunnYu/mish-cuda
cd /
git clone https://github.com/JunnYu/mish-cuda
cd mish-cuda
python setup.py build install

# go to code folder
cd /yolo
```

local:

```
pip install -r requirements.txt
```

※ For running Mish models, please install <https://github.com/thomasbrandon/mish-cuda>

Training

```
python train.py --device 0 --batch-size 16 --img 640 640 --data coco.yaml -
-cfg cfg/yolov4-pacsp.cfg --weights '' --name yolov4-pacsp
```

Testing

```
python test.py --img 640 --conf 0.001 --batch 8 --device 0 --data coco.yaml
--cfg cfg/yolov4-pacsp.cfg --weights weights/yolov4-pacsp.pt
```

Citation

```
@article{bochkovskiy2020yolov4,
  title={YOLOv4: Optimal Speed and Accuracy of Object Detection},
```

```
author={Bochkovskiy, Alexey and Wang, Chien-Yao and Liao, Hong-Yuan Mark},  
journal={arXiv preprint arXiv:2004.10934},  
year={2020}  
}
```

```
@inproceedings{wang2020cspnet,  
  title={{CSPNet}: A New Backbone That Can Enhance Learning Capability of {CNN}},  
  author={Wang, Chien-Yao and Mark Liao, Hong-Yuan and Wu, Yueh-Hua and Chen, Ping-Yang and Hsieh, Jun-Wei and Yeh, I-Hau},  
  booktitle={Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops},  
  pages={390--391},  
  year={2020}  
}
```

Acknowledgements

- <https://github.com/AlexeyAB/darknet>
- <https://github.com/ultralytics/yolov3>
- <https://github.com/ultralytics/yolov5>

Official YOLOv7

Implementation of paper - [YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors](#)

[custom badge](#) [unparseable json response](#)[🤗 Hugging Face Spaces](#)[🔗 Open in Colab](#)[cs.CV](#)[arXiv:2207.02696](#)

Web Demo

- Integrated into [Huggingface Spaces](#) 🤗 using Gradio. Try out the Web Demo [🤗 Hugging Face Spaces](#)

Performance

MS COCO

Model	Test Size	AP ^{test}	AP ₅₀ ^{test}	AP ₇₅ ^{test}	batch 1 fps	batch 32 average time
YOLOv7	640	51.4%	69.7%	55.9%	161 <i>fps</i>	2.8 <i>ms</i>
YOLOv7-X	640	53.1%	71.2%	57.8%	114 <i>fps</i>	4.3 <i>ms</i>
YOLOv7-W6	1280	54.9%	72.6%	60.1%	84 <i>fps</i>	7.6 <i>ms</i>
YOLOv7-E6	1280	56.0%	73.5%	61.2%	56 <i>fps</i>	12.3 <i>ms</i>
YOLOv7-D6	1280	56.6%	74.0%	61.8%	44 <i>fps</i>	15.0 <i>ms</i>
YOLOv7-E6E	1280	56.8%	74.4%	62.1%	36 <i>fps</i>	18.7 <i>ms</i>

Installation

Docker environment (recommended)

► Expand

```
# create the docker container, you can change the share memory size if you
have more.
nvidia-docker run --name yolov7 -it -v your_coco_path/./coco/ -v
your_code_path/./yolov7 --shm-size=64g nvcr.io/nvidia/pytorch:21.08-py3

# apt install required packages
apt update
apt install -y zip htop screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop
```

```
# go to code folder
cd /yolov7
```

Testing

yolov7.pt yolov7x.pt yolov7-w6.pt yolov7-e6.pt yolov7-d6.pt yolov7-e6e.pt

```
python test.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --
iou 0.65 --device 0 --weights yolov7.pt --name yolov7_640_val
```

You will get the results:

```

Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] =
0.51206
Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] =
0.69730
Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] =
0.55521
Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] =
0.35247
Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] =
0.55937
Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] =
0.66693
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] =
0.38453
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] =
0.63765
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] =
0.68772
Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] =
0.53766
Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] =
0.73549
Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] =
0.83868
```

To measure accuracy, download [COCO-annotations for Pycocotools](#) to the
./coco/annotations/instances_val2017.json

Training

Data preparation

```
bash scripts/get_coco.sh
```

- Download MS COCO dataset images ([train](#), [val](#), [test](#)) and [labels](#). If you have previously used a different version of YOLO, we strongly recommend that you delete `train2017.cache` and `val2017.cache` files, and redownload [labels](#)

Single GPU training

```
# train p5 models
python train.py --workers 8 --device 0 --batch-size 32 --data
data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --
name yolov7 --hyp data/hyp.scratch.p5.yaml

# train p6 models
python train_aux.py --workers 8 --device 0 --batch-size 16 --data
data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml --weights
'' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

Multiple GPU training

```
# train p5 models
python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527
train.py --workers 8 --device 0,1,2,3 --sync-bn --batch-size 128 --data
data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --
name yolov7 --hyp data/hyp.scratch.p5.yaml

# train p6 models
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527
train_aux.py --workers 8 --device 0,1,2,3,4,5,6,7 --sync-bn --batch-size
128 --data data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml
--weights '' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

Transfer learning

[yolov7_training.pt](#) [yolov7x_training.pt](#) [yolov7-w6_training.pt](#) [yolov7-e6_training.pt](#)
[yolov7-d6_training.pt](#) [yolov7-e6e_training.pt](#)

Single GPU finetuning for custom dataset

```
# finetune p5 models
python train.py --workers 8 --device 0 --batch-size 32 --data
data/custom.yaml --img 640 640 --cfg cfg/training/yolov7-custom.yaml --
weights 'yolov7_training.pt' --name yolov7-custom --hyp
data/hyp.scratch.custom.yaml

# finetune p6 models
python train_aux.py --workers 8 --device 0 --batch-size 16 --data
data/custom.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6-custom.yaml -
-weights 'yolov7-w6_training.pt' --name yolov7-w6-custom --hyp
data/hyp.scratch.custom.yaml
```

Re-parameterization

See [reparameterization.ipynb](#)

Inference

On video:

```
python detect.py --weights yolov7.pt --conf 0.25 --img-size 640 --source
yourvideo.mp4
```

On image:

```
python detect.py --weights yolov7.pt --conf 0.25 --img-size 640 --source
inference/images/horses.jpg
```



Export

Pytorch to CoreML (and inference on MacOS/iOS)  [Open in Colab](#)

Pytorch to ONNX with NMS (and inference)  [Open in Colab](#)

```
python export.py --weights yolov7-tiny.pt --grid --end2end --simplify \
    --topk-all 100 --iou-thres 0.65 --conf-thres 0.35 --img-size 640
640 --max-wh 640
```

Pytorch to TensorRT with NMS (and inference)  [Open in Colab](#)

```
wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-
tiny.pt
python export.py --weights ./yolov7-tiny.pt --grid --end2end --simplify --
topk-all 100 --iou-thres 0.65 --conf-thres 0.35 --img-size 640 640
git clone https://github.com/Linaom1214/tensorrt-python.git
python ./tensorrt-python/export.py -o yolov7-tiny.onnx -e yolov7-tiny-
nms.trt -p fp16
```

Pytorch to TensorRT another way  [Open in Colab](#)

► **Expand**

YOLOv9

Implementation of paper - [YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information](#)

cs.CV

arXiv:2402.13616

 Hugging Face

Spaces

 Hugging Face

Spaces

 Open in Colab

 OpenCV

BlogPost



Performance

MS COCO

Model	Test Size	AP ^{val}	AP ₅₀ ^{val}	AP ₇₅ ^{val}	Param.	FLOPs
YOLOv9-T	640	38.3%	53.1%	41.3%	2.0M	7.7G
YOLOv9-S	640	46.8%	63.4%	50.7%	7.1M	26.4G
YOLOv9-M	640	51.4%	68.1%	56.1%	20.0M	76.3G
YOLOv9-C	640	53.0%	70.2%	57.8%	25.3M	102.1G
YOLOv9-E	640	55.6%	72.8%	60.6%	57.3M	189.0G
<!--	YOLOv9 (ReLU)	640	51.9%	69.1%	56.5%	25.3M

Useful Links

► Expand

- Custom training: <https://github.com/WongKinYiu/yolov9/issues/30#issuecomment-1960955297>
- ONNX export: <https://github.com/WongKinYiu/yolov9/issues/2#issuecomment-1960519506>
<https://github.com/WongKinYiu/yolov9/issues/40#issue-2150697688> <https://github.com/WongKinYiu/yolov9/issues/130#issue-2162045461>
- ONNX export for segmentation: <https://github.com/WongKinYiu/yolov9/issues/260#issue-2191162150>
- TensorRT inference: <https://github.com/WongKinYiu/yolov9/issues/143#issuecomment-1975049660>
<https://github.com/WongKinYiu/yolov9/issues/34#issue-2150393690> <https://github.com/WongKinYiu/yolov9/issues/79#issue-2153547004>
<https://github.com/WongKinYiu/yolov9/issues/143#issue-2164002309>
- QAT TensorRT: <https://github.com/WongKinYiu/yolov9/issues/327#issue-2229284136> <https://github.com/WongKinYiu/yolov9/issues/253#issue-2189520073>
- TensorRT inference for segmentation: <https://github.com/WongKinYiu/yolov9/issues/446>
- TFLite: <https://github.com/WongKinYiu/yolov9/issues/374#issuecomment-2065751706>
- OpenVINO: <https://github.com/WongKinYiu/yolov9/issues/164#issue-2168540003>
- C# ONNX inference: <https://github.com/WongKinYiu/yolov9/issues/95#issue-2155974619>
- C# OpenVINO inference: <https://github.com/WongKinYiu/yolov9/issues/95#issuecomment-1968131244>
- OpenCV: <https://github.com/WongKinYiu/yolov9/issues/113#issuecomment-1971327672>
- Hugging Face demo: <https://github.com/WongKinYiu/yolov9/issues/45#issuecomment-1961496943>
- CoLab demo: <https://github.com/WongKinYiu/yolov9/pull/18>
- ONNXSlim export: <https://github.com/WongKinYiu/yolov9/pull/37>
- YOLOv9 ROS: <https://github.com/WongKinYiu/yolov9/issues/144#issue-2164210644>
- YOLOv9 ROS TensorRT: <https://github.com/WongKinYiu/yolov9/issues/145#issue-2164218595>
- YOLOv9 Julia: <https://github.com/WongKinYiu/yolov9/issues/141#issuecomment-1973710107>
- YOLOv9 MLX: <https://github.com/WongKinYiu/yolov9/issues/258#issue-2190586540>
- YOLOv9 StrongSORT with OSNet: <https://github.com/WongKinYiu/yolov9/issues/299#issue-2212093340>
- YOLOv9 ByteTrack: <https://github.com/WongKinYiu/yolov9/issues/78#issue-2153512879>

YOLOv9 DeepSORT: <https://github.com/WongKinYiu/yolov9/issues/98#issue-2156172319>

YOLOv9 counting: <https://github.com/WongKinYiu/yolov9/issues/84#issue-2153904804>

YOLOv9 speed estimation: <https://github.com/WongKinYiu/yolov9/issues/456>

YOLOv9 face detection: <https://github.com/WongKinYiu/yolov9/issues/121#issue-2160218766>

YOLOv9 segmentation onnxruntime: <https://github.com/WongKinYiu/yolov9/issues/151#issue-2165667350>

Comet logging: <https://github.com/WongKinYiu/yolov9/pull/110>

MLflow logging: <https://github.com/WongKinYiu/yolov9/pull/87>

AnyLabeling tool: <https://github.com/WongKinYiu/yolov9/issues/48#issue-2152139662>

AX650N deploy: <https://github.com/WongKinYiu/yolov9/issues/96#issue-2156115760>

Conda environment: <https://github.com/WongKinYiu/yolov9/pull/93>

AutoDL docker environment: <https://github.com/WongKinYiu/yolov9/issues/112#issue-2158203480>

Installation

Docker environment (recommended)

► Expand

```
# create the docker container, you can change the share memory size if you have more.
nvidia-docker run --name yolov9 -it -v your_coco_path:/coco/ -v your_code_path:/yolov9 --shm-size=64g
nvcr.io/nvidia/pytorch:21.11-py3

# apt install required packages
apt update
apt install -y zip htop screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop

# go to code folder
cd /yolov9
```

Evaluation

yolov9-s-converted.pt yolov9-m-converted.pt yolov9-c-converted.pt yolov9-e-converted.pt yolov9-s.pt yolov9-m.pt
yolov9-c.pt yolov9-e.pt gelan-s.pt gelan-m.pt gelan-c.pt gelan-e.pt

```
# evaluate converted yolov9 models
python val.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --weights
'./yolov9-c-converted.pt' --save-json --name yolov9_c_c_640_val

# evaluate yolov9 models
# python val_dual.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --
weights './yolov9-c.pt' --save-json --name yolov9_c_640_val

# evaluate gelan models
# python val.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --weights
'./gelan-c.pt' --save-json --name gelan_c_640_val
```

You will get the results:

```
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.530
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.702
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.578
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.362
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.585
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.693
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.392
```

```

Average Recall      (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.652
Average Recall      (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.702
Average Recall      (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.541
Average Recall      (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.760
Average Recall      (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.844

```

Training

Data preparation

```
bash scripts/get_coco.sh
```

- Download MS COCO dataset images ([train](#), [val](#), [test](#)) and [labels](#). If you have previously used a different version of YOLO, we strongly recommend that you delete [train2017.cache](#) and [val2017.cache](#) files, and redownload [labels](#)

Single GPU training

```

# train yolov9 models
python train_dual.py --workers 8 --device 0 --batch 16 --data data/coco.yaml --img 640 --cfg
models/detect/yolov9-c.yaml --weights '' --name yolov9-c --hyp hyp.scratch-high.yaml --min-items 0 --
epochs 500 --close-mosaic 15

# train gelan models
# python train.py --workers 8 --device 0 --batch 32 --data data/coco.yaml --img 640 --cfg
models/detect/gelan-c.yaml --weights '' --name gelan-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs
500 --close-mosaic 15

```

Multiple GPU training

```

# train yolov9 models
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 train_dual.py --workers 8 --
device 0,1,2,3,4,5,6,7 --sync-bn --batch 128 --data data/coco.yaml --img 640 --cfg models/detect/yolov9-
c.yaml --weights '' --name yolov9-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs 500 --close-mosaic
15

# train gelan models
# python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527 train.py --workers 8 --device
0,1,2,3 --sync-bn --batch 128 --data data/coco.yaml --img 640 --cfg models/detect/gelan-c.yaml --weights
'' --name gelan-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs 500 --close-mosaic 15

```

Re-parameterization

See [reparameterization.ipynb](#).

Inference



```

# inference converted yolov9 models
python detect.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './yolov9-c-
converted.pt' --name yolov9_c_c_640_detect

# inference yolov9 models
# python detect_dual.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './yolov9-c.pt'
--name yolov9_c_640_detect

# inference gelan models
# python detect.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './gelan-c.pt' --
name gelan_c_c_640_detect

```

Citation

```
@article{wang2024yolov9,
  title={{YOLOv9}: Learning What You Want to Learn Using Programmable Gradient Information},
  author={Wang, Chien-Yao and Liao, Hong-Yuan Mark},
  booktitle={arXiv preprint arXiv:2402.13616},
  year={2024}
}
```

```
@article{chang2023yolor,
  title={{YOLOR}-Based Multi-Task Learning},
  author={Chang, Hung-Shuo and Wang, Chien-Yao and Wang, Richard Robert and Chou, Gene and Liao, Hong-Yuan Mark},
  journal={arXiv preprint arXiv:2309.16921},
  year={2023}
}
```

Teaser

Parts of code of [YOLOR-Based Multi-Task Learning](#) are released in the repository.



Object Detection

gelan-c-det.pt

object detection

```
# coco/labels/{split}/*.txt
# bbox or polygon (1 instance 1 line)
python train.py --workers 8 --device 0 --batch 32 --data data/coco.yaml --img 640 --cfg
models/detect/gelan-c.yaml --weights '' --name gelan-c-det --hyp hyp.scratch-high.yaml --min-items 0 --
epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	Ap ^{box}
GELAN-C-DET	640	25.3M	102.1G	52.3%
YOLOv9-C-DET	640	25.3M	102.1G	53.0%

Instance Segmentation

gelan-c-seg.pt

object detection instance segmentation

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
python segment/train.py --workers 8 --device 0 --batch 32 --data coco.yaml --img 640 --cfg
models/segment/gelan-c-seg.yaml --weights '' --name gelan-c-seg --hyp hyp.scratch-high.yaml --no-overlap -
-epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	Ap ^{box}	Ap ^{mask}
GELAN-C-SEG	640	27.4M	144.6G	52.3%	42.4%
YOLOv9-C-SEG	640	27.4M	145.5G	53.3%	43.5%

Panoptic Segmentation

gelan-c-pan.pt

object detection instance segmentation semantic segmentation stuff segmentation panoptic segmentation

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
# coco/stuff/{split}/*.txt
# polygon (1 semantic 1 line)
python panoptic/train.py --workers 8 --device 0 --batch 32 --data coco.yaml --img 640 --cfg
models/panoptic/gelan-c-pan.yaml --weights '' --name gelan-c-pan --hyp hyp.scratch-high.yaml --no-overlap
--epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP ^{box}	AP ^{mask}	mIoU _{164k/10k} ^{semantic}	mIoU ^{stuff}	PQ ^{panoptic}
GELAN-C-PAN	640	27.6M	146.7G	52.6%	42.5%	39.0%/48.3%	52.7%	39.4%
YOLOv9-C-PAN	640	28.8M	187.0G	52.7%	43.0%	39.8%/-	52.2%	40.5%

Image Captioning (not yet released)

object detection instance segmentation semantic segmentation stuff segmentation panoptic segmentation image captioning

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
# coco/stuff/{split}/*.txt
# polygon (1 semantic 1 line)
# coco/annotations/*.json
# json (1 split 1 file)
python caption/train.py --workers 8 --device 0 --batch 32 --data coco.yaml --img 640 --cfg
models/caption/gelan-c-cap.yaml --weights '' --name gelan-c-cap --hyp hyp.scratch-high.yaml --no-overlap -
-epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP ^{box}	AP ^{mask}	mIoU _{164k/10k} ^{semantic}	mIoU ^{stuff}	PQ ^{panoptic}	BLEU@4 ^{caption}	CIDEr ^{caption}
GELAN-C-CAP	640	47.5M	-	51.9%	42.6%	42.5%/-	56.5%	41.7%	38.8	122.3
YOLOv9-C-CAP	640	47.5M	-	52.1%	42.6%	43.0%/-	56.4%	42.1%	39.1	122.0
<!-- YOLOR-MT	640	79.3M	-	51.0%	41.7%	-/49.6%	55.9%	40.5%	35.7	

Acknowledgements

► Expand

- <https://github.com/AlexeyAB/darknet>
- <https://github.com/WongKinYiu/yolor>
- <https://github.com/WongKinYiu/yolov7>
- <https://github.com/VDIGPKU/DynamicDet>
- <https://github.com/DingXiaoH/RepVGG>
- <https://github.com/ultralytics/yolov5>
- <https://github.com/meituan/YOLOv6>

GeneralistYOLO

Generalist YOLO: Towards Real-Time End-to-End Multi-Task Visual Language Models



Performance

Generalist Visual Language Tasks

MS COCO

Model	Size	#Param.	FLOPs	AP _{e2e/nms} ^{box}	AP _{e2e/nms} ^{mask}	mIoU _{164k/10k} ^{semantic}	mIoU ^{stuff}	PQ ^{panoptic}	BLEU@4 ^{caption}	CIDEr ^{caption}
GeneralistYOLO	640	32.5M	~122.2G	52.4%/52.8%	43.0%/43.1%	44.7%/52.0%	59.2%	44.2%	38.7	122.7
GYOLO	640	32.5M	~122.2G	52.3%/	42.9%/	44.5%/	59.2%	44.3%	38.9	122.8
GYOLO-X	640	-	-	53.3%/	43.8%/	46.0%/	59.9%	45.4%	39.2	123.5
<!--	****		M	G	%/%	%/%	%/%	%	%	****

Installation

Generalist Visual Language Tasks

Docker environment (recommended)

► expand

```
# create the docker container, you can change the share memory size if you have more.
nvidia-docker run --name gyolo -it -v your_coco_path:/coco/ -v your_code_path:/gyolo --shm-size=64g
nvcr.io/nvidia/pytorch:21.11-py3
```

Data preparation

► expand

Packages

```
pip install -r requirements.txt
```

Note: java is required for pycocoevalcap tool

Dataset

We use [MS COCO Dataset](#).

Please download the following files:

- Images
 - 2017 Train images
 - 2017 Val images
- Annotations
 - 2017 Train/Val annotations
 - 2017 Stuff Train/Val annotations
 - 2017 Panoptic Train/Val annotations

Labels

Download the labels from:

- [COCO](#)
- [Segmentation labels](#) Extract these zip files and put the data of **Segmentation labels** into **coco/stuff** folder.

Modify Yaml File

Modify path of **data/coco.yaml** to use the dataset:

```
path: path_to_coco_dir
train: train2017.txt
val: val2017.txt
```

Dataset Folder Hierarchy

The folder hierarchy should be:

```
coco/
├── annotations/
│   ├── captions_train2017.json
│   ├── captions_val2017.json
│   ├── instances_train2017.json
│   ├── instances_val2017.json
│   ├── panoptic_train2017.json
│   ├── panoptic_val2017.json
│   ├── stuff_train2017.json
│   └── stuff_val2017.json
├── images/
│   ├── train2017/
│   │   └── *.jpg
│   └── val2017/
│       └── *.jpg
├── labels/
│   ├── train2017/
│   │   └── *.txt
│   └── val2017/
│       └── *.txt
├── panoptic/
│   └── masks/
│       ├── train2017/
│       │   └── *.png
│       └── val2017/
│           └── *.png
├── stuff/
│   ├── train2017/
│   │   └── *.txt
│   └── val2017/
│       └── *.txt
├── train2017.txt
└── val2017.txt
```

Evaluation

Generalist Visual Language Tasks

```
python caption/val.py --data coco.yaml --batch 4 --weights 'gyolo.pt' --img 640 --conf 0.001 --iou 0.7 --device 0 --
save-json --export-mask
```

► End-to-End Performance

Object Detection

Average Precision	(AP) @[IoU=0.50:0.95 area=	all maxDets=100]	= 0.524
Average Precision	(AP) @[IoU=0.50 area=	all maxDets=100]	= 0.700
Average Precision	(AP) @[IoU=0.75 area=	all maxDets=100]	= 0.569
Average Precision	(AP) @[IoU=0.50:0.95 area=	small maxDets=100]	= 0.343
Average Precision	(AP) @[IoU=0.50:0.95 area=	medium maxDets=100]	= 0.580
Average Precision	(AP) @[IoU=0.50:0.95 area=	large maxDets=100]	= 0.680
Average Recall	(AR) @[IoU=0.50:0.95 area=	all maxDets= 1]	= 0.388
Average Recall	(AR) @[IoU=0.50:0.95 area=	all maxDets= 10]	= 0.646
Average Recall	(AR) @[IoU=0.50:0.95 area=	all maxDets=100]	= 0.695
Average Recall	(AR) @[IoU=0.50:0.95 area=	small maxDets=100]	= 0.529
Average Recall	(AR) @[IoU=0.50:0.95 area=	medium maxDets=100]	= 0.754
Average Recall	(AR) @[IoU=0.50:0.95 area=	large maxDets=100]	= 0.843

Instance Segmentation

Average Precision	(AP) @[IoU=0.50:0.95 area=	all maxDets=100]	= 0.430
Average Precision	(AP) @[IoU=0.50 area=	all maxDets=100]	= 0.669
Average Precision	(AP) @[IoU=0.75 area=	all maxDets=100]	= 0.457

```

Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.232
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.480
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.611
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.337
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.538
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.571
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.365
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.633
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.766

```

Semantic Segmentation (164k)

Class	IoU	Acc
unlabeled	nan	nan
person	84.86	93.16
bicycle	70.03	85.5
car	64.03	80.68
motorcycle	82.11	92.03
airplane	83.58	95.03
bus	83.89	88.31
train	85.93	95.77
truck	58.29	77.87
boat	60.76	80.73
traffic light	72.42	88.09
fire hydrant	87.81	96.83
stop sign	88.85	98.47
parking meter	71.55	79.2
bench	48.87	72.44
bird	70.25	80.34
cat	85.84	94.42
dog	77.41	86.99
horse	82.46	96.0
sheep	86.14	95.76
cow	84.38	91.59
elephant	90.21	97.53
bear	89.74	94.42
zebra	88.84	95.87
giraffe	86.82	95.68
backpack	29.63	53.67
umbrella	74.51	87.73
handbag	26.91	46.98
tie	1.52	7.45
suitcase	72.03	88.68
frisbee	33.48	88.33
skis	48.53	63.79
snowboard	54.1	62.81
sports ball	61.87	74.01
kite	54.46	71.27
baseball bat	44.28	63.32
baseball glove	45.31	75.14
skateboard	59.39	69.72
surfboard	69.36	82.31
tennis racket	77.2	94.05
bottle	60.69	76.84
wine glass	71.92	83.25
cup	56.67	78.08
fork	50.22	59.83
knife	36.22	45.91
spoon	38.49	48.59
bowl	52.75	64.88
banana	74.19	90.02
apple	46.75	66.49
sandwich	52.52	73.81
orange	62.68	90.65
broccoli	47.92	69.18
carrot	55.95	72.26
hot dog	62.78	71.53
pizza	80.19	90.89
donut	62.88	81.4
cake	63.41	80.72
chair	48.88	70.26
couch	52.75	81.58
potted plant	32.8	52.98
bed	62.34	80.18
dining table	46.45	77.74
toilet	80.01	93.89
tv	70.21	85.8

	laptop	76.01	92.74	
	mouse	71.09	90.58	
	remote	41.18	74.18	
	keyboard	76.8	86.66	
	cell phone	69.47	88.76	
	microwave	71.89	80.98	
	oven	57.07	85.7	
	toaster	16.02	19.88	
	sink	54.85	81.12	
	refrigerator	72.63	91.97	
	book	43.71	67.28	
	clock	66.02	81.57	
	vase	57.18	81.76	
	scissors	57.41	68.47	
	teddy bear	75.06	94.48	
	hair drier	12.1	13.0	
	toothbrush	42.57	82.4	
	banner	28.75	64.1	
	blanket	4.71	4.9	
	branch	17.54	19.07	
	bridge	29.11	41.19	
	building-other	53.07	71.46	
	bush	28.35	35.92	
	cabinet	53.94	73.32	
	cage	22.65	37.39	
	cardboard	38.95	52.3	
	carpet	48.63	70.33	
	ceiling-other	59.97	83.41	
	ceiling-tile	0.0	0.0	
	cloth	0.0	0.0	
	clothes	15.54	19.18	
	clouds	51.44	70.78	
	counter	27.17	49.03	
	cupboard	0.0	0.0	
	curtain	61.09	75.68	
	desk-stuff	41.19	58.88	
	dirt	39.18	63.3	
	door-stuff	34.88	58.0	
	fence	32.11	55.61	
	floor-marble	0.0	0.0	
	floor-other	20.38	28.56	
	floor-stone	1.22	1.45	
	floor-tile	59.88	73.49	
	floor-wood	56.31	76.05	
	flower	41.24	63.94	
	fog	5.29	5.53	
	food-other	23.03	29.74	
	fruit	23.25	33.46	
	furniture-other	11.95	14.54	
	grass	67.63	84.0	
	gravel	24.22	33.02	
	ground-other	0.01	0.01	
	hill	11.37	14.06	
	house	24.15	28.92	
	leaves	19.67	23.16	
	light	33.13	43.04	
	mat	0.0	0.0	
	metal	25.95	35.11	
	mirror-stuff	46.3	59.93	
	moss	0.0	0.0	
	mountain	51.84	69.33	
	mud	0.0	0.0	
	napkin	0.0	0.0	
	net	38.73	56.65	
	paper	26.97	37.97	
	pavement	47.96	65.6	
	pillow	0.04	0.04	
	plant-other	11.92	17.66	
	plastic	8.67	10.36	
	platform	26.13	43.2	
	playingfield	67.44	85.82	
	railing	0.0	0.0	
	railroad	58.88	80.49	
	river	47.47	73.29	
	road	63.05	85.34	
	rock	49.4	78.22	
	roof	13.73	16.23	
	rug	31.78	47.12	
	salad	0.0	0.0	
	sand	59.8	66.58	
	sea	84.48	93.09	

	shelf		28.05		44.77	
	sky-other		71.52		85.74	
	skyscraper		34.83		45.86	
	snow		87.85		94.18	
	solid-other		0.0		0.0	
	stairs		18.45		34.88	
	stone		0.04		0.04	
	straw		25.23		35.38	
	structural-other		0.0		0.0	
	table		21.97		32.77	
	tent		7.99		10.02	
	textile-other		11.05		16.53	
	towel		26.33		34.97	
	tree		71.69		89.39	
	vegetable		29.58		39.57	
	wall-brick		44.48		65.1	
	wall-concrete		56.56		77.85	
	wall-other		17.42		26.48	
	wall-panel		0.0		0.0	
	wall-stone		28.92		35.3	
	wall-tile		62.57		81.9	
	wall-wood		36.82		52.74	
	water-other		24.21		32.21	
	waterdrops		0.0		0.0	
	window-blind		43.78		55.65	
	window-other		41.27		68.13	
	wood		14.98		20.71	
	other		nan		nan	
+-----+-----+-----+						
mIoU: 44.71						

Sementic Segmentation (10k)

+-----+-----+-----+						
	Class		IoU		Acc	
+-----+-----+-----+						
	unlabeled		0.0		0.0	
	person		89.51		93.95	
	bicycle		80.23		94.89	
	car		77.79		91.13	
	motorcycle		91.53		96.09	
	airplane		91.56		96.86	
	bus		78.32		82.81	
	train		84.03		97.35	
	truck		81.86		91.39	
	boat		82.03		91.1	
	traffic light		83.19		92.57	
	fire hydrant		94.32		97.83	
	stop sign		96.3		98.8	
	parking meter		93.45		98.75	
	bench		63.4		73.69	
	bird		78.83		83.16	
	cat		92.11		96.2	
	dog		90.22		96.16	
	horse		91.82		96.67	
	sheep		84.12		88.55	
	cow		95.46		98.8	
	elephant		91.32		97.54	
	bear		94.91		98.01	
	zebra		91.93		96.91	
	giraffe		88.71		95.79	
	backpack		26.41		64.16	
	umbrella		74.86		82.87	
	handbag		19.46		28.59	
	tie		56.53		87.15	
	suitcase		78.7		96.52	
	frisbee		94.25		98.44	
	skis		53.84		71.81	
	snowboard		78.0		89.26	
	sports ball		80.14		92.07	
	kite		82.64		90.41	
	baseball bat		61.12		88.1	
	baseball glove		87.25		96.25	
	skateboard		71.44		92.04	
	surfboard		92.07		95.83	
	tennis racket		86.32		98.22	
	bottle		62.63		67.93	
	wine glass		83.16		94.68	
	cup		74.92		84.94	

fork	42.35	61.53
knife	75.21	79.47
spoon	53.88	59.72
bowl	65.96	70.1
banana	84.71	94.33
apple	73.57	84.38
sandwich	87.26	88.74
orange	68.71	91.81
broccoli	87.19	94.84
carrot	73.76	84.37
hot dog	48.05	96.32
pizza	93.58	95.03
donut	62.73	73.17
cake	91.87	93.91
chair	66.2	87.91
couch	77.24	95.24
potted plant	45.71	56.18
bed	81.64	92.91
dining table	66.72	88.02
toilet	92.97	97.25
tv	78.62	96.39
laptop	88.94	97.86
mouse	75.67	79.79
remote	51.58	62.81
keyboard	90.94	95.07
cell phone	84.11	97.22
microwave	41.51	76.53
oven	73.16	88.12
toaster	19.62	19.62
sink	84.92	89.03
refrigerator	87.58	97.52
book	77.65	90.83
clock	74.15	92.76
vase	68.39	96.87
scissors	82.61	96.54
teddy bear	87.25	98.11
hair drier	0.94	0.94
toothbrush	25.65	30.9
banner	40.85	63.5
blanket	0.0	0.0
branch	0.0	0.0
bridge	8.37	10.71
building-other	61.54	80.41
bush	27.83	30.96
cabinet	21.48	79.41
cage	0.0	0.0
cardboard	13.07	23.45
carpet	64.81	86.65
ceiling-other	64.85	86.37
ceiling-tile	0.0	0.0
cloth	0.0	0.0
clothes	33.63	45.14
clouds	40.58	45.55
counter	52.43	59.55
cupboard	0.0	0.0
curtain	66.88	81.6
desk-stuff	37.35	49.24
dirt	36.32	70.93
door-stuff	49.48	68.39
fence	41.34	57.21
floor-marble	0.0	0.0
floor-other	35.03	58.93
floor-stone	14.11	21.97
floor-tile	61.11	73.52
floor-wood	66.94	85.74
flower	20.39	54.75
fog	0.0	0.0
food-other	30.71	43.36
fruit	65.3	84.93
furniture-other	17.89	20.81
grass	75.38	89.23
gravel	29.83	35.46
ground-other	0.0	0.0
hill	26.77	29.34
house	41.49	52.24
leaves	46.16	59.14
light	34.34	43.4
mat	0.0	0.0
metal	12.79	30.98
mirror-stuff	53.17	62.22
moss	0.0	0.0

	mountain		36.63		91.39	
	mud		0.0		0.0	
	napkin		0.0		0.0	
	net		52.53		57.16	
	paper		29.87		56.91	
	pavement		48.03		85.76	
	pillow		0.0		0.0	
	plant-other		27.87		30.27	
	plastic		16.85		25.25	
	platform		49.37		55.79	
	playingfield		60.95		69.98	
	railing		0.05		0.05	
	railroad		56.27		89.34	
	river		47.54		92.76	
	road		76.87		86.12	
	rock		57.34		81.09	
	roof		25.76		34.79	
	rug		58.84		68.68	
	salad		0.0		0.0	
	sand		63.62		79.03	
	sea		75.81		93.63	
	shelf		18.32		44.97	
	sky-other		62.14		88.1	
	skyscraper		57.28		68.92	
	snow		88.33		94.69	
	solid-other		0.0		0.0	
	stairs		46.39		67.51	
	stone		0.0		0.0	
	straw		15.59		33.19	
	structural-other		0.0		0.0	
	table		20.29		33.7	
	tent		65.86		82.32	
	textile-other		17.8		25.05	
	towel		40.82		42.54	
	tree		78.66		93.6	
	vegetable		47.9		68.54	
	wall-brick		52.22		68.49	
	wall-concrete		7.76		84.44	
	wall-other		10.07		10.59	
	wall-panel		0.0		0.0	
	wall-stone		40.65		49.9	
	wall-tile		61.45		94.53	
	wall-wood		41.37		63.97	
	water-other		14.65		14.94	
	waterdrops		0.0		0.0	
	window-blind		34.87		71.5	
	window-other		48.62		62.69	
	wood		12.66		21.92	
+-----+-----+-----+-----+						
{ 'aAcc': 73.17, 'mIoU': 52.0, 'mAcc': 64.1 }						

Stuff Segmentation

Mean IOU	@[classes=	leaves]	=	0.3007
FW IOU	@[classes=	leaves]	=	0.5923
Mean accuracy	@[classes=	leaves]	=	0.4080
Pixel accuracy	@[classes=	leaves]	=	0.7328
Mean IOU	@[classes=supercats	]	=	0.5855
FW IOU	@[classes=supercats	]	=	0.7167
Mean accuracy	@[classes=supercats	]	=	0.7039
Pixel accuracy	@[classes=supercats	]	=	0.8304
class	class name		iou	macc
92	banner		0.2896	0.6410
93	blanket		0.0471	0.0490
94	branch		0.1754	0.1907
95	bridge		0.2911	0.4119
96	building-other		0.5348	0.7146
97	bush		0.2836	0.3592
98	cabinet		0.5399	0.7332
99	cage		0.2267	0.3739
100	cardboard		0.3905	0.5230
101	carpet		0.4863	0.7033
102	ceiling-other		0.6000	0.8341
103	ceiling-tile		0.0000	0.0000
104	cloth		0.0000	0.0000
105	clothes		0.1589	0.1918
106	clouds		0.5148	0.7078
107	counter		0.2717	0.4903
108	cupboard		0.0000	0.0000

109	curtain	0.6111	0.7568
110	desk-stuff	0.4120	0.5888
111	dirt	0.3920	0.6330
112	door-stuff	0.3492	0.5800
113	fence	0.3232	0.5561
114	floor-marble	0.0000	0.0000
115	floor-other	0.2041	0.2856
116	floor-stone	0.0122	0.0145
117	floor-tile	0.5990	0.7349
118	floor-wood	0.5633	0.7605
119	flower	0.4130	0.6394
120	fog	0.0529	0.0553
121	food-other	0.2344	0.2974
122	fruit	0.2396	0.3346
123	furniture-other	0.1198	0.1454
124	grass	0.6778	0.8400
125	gravel	0.2422	0.3302
126	ground-other	0.0001	0.0001
127	hill	0.1138	0.1406
128	house	0.2418	0.2892
129	leaves	0.1968	0.2316
130	light	0.3313	0.4304
131	mat	0.0000	0.0000
132	metal	0.2600	0.3511
133	mirror-stuff	0.4630	0.5993
134	moss	0.0000	0.0000
135	mountain	0.5194	0.6933
136	mud	0.0000	0.0000
137	napkin	0.0000	0.0000
138	net	0.3873	0.5665
139	paper	0.2706	0.3797
140	pavement	0.4802	0.6560
141	pillow	0.0004	0.0004
142	plant-other	0.1192	0.1766
143	plastic	0.0867	0.1036
144	platform	0.2613	0.4320
145	playingfield	0.6762	0.8582
146	railing	0.0000	0.0000
147	railroad	0.5888	0.8049
148	river	0.4772	0.7329
149	road	0.6336	0.8534
150	rock	0.4942	0.7822
151	roof	0.1374	0.1623
152	rug	0.3178	0.4712
153	salad	0.0000	0.0000
154	sand	0.5991	0.6658
155	sea	0.8463	0.9309
156	shelf	0.2912	0.4477
157	sky-other	0.7164	0.8574
158	skyscraper	0.3484	0.4586
159	snow	0.8814	0.9418
160	solid-other	0.0000	0.0000
161	stairs	0.1998	0.3488
162	stone	0.0004	0.0004
163	straw	0.2523	0.3538
164	structural-other	0.0000	0.0000
165	table	0.2199	0.3277
166	tent	0.0801	0.1002
167	textile-other	0.1107	0.1653
168	towel	0.2633	0.3497
169	tree	0.7186	0.8939
170	vegetable	0.2972	0.3957
171	wall-brick	0.4457	0.6510
172	wall-concrete	0.5664	0.7785
173	wall-other	0.1750	0.2648
174	wall-panel	0.0000	0.0000
175	wall-stone	0.2892	0.3530
176	wall-tile	0.6257	0.8190
177	wall-wood	0.3684	0.5274
178	water-other	0.2423	0.3221
179	waterdrops	0.0000	0.0000
180	window-blind	0.4380	0.5565
181	window-other	0.4135	0.6813
182	wood	0.1498	0.2071
183	other	0.8158	0.9459

	PQ	SQ	RQ	N
All	44.2	81.3	52.8	133
Things	53.0	83.8	62.6	80
Stuff	30.9	77.4	38.1	53

Image Captioning

```

Bleu_1: 0.783
Bleu_2: 0.638
Bleu_3: 0.500
Bleu_4: 0.387
METEOR: 0.279
ROUGE_L: 0.584
CIDER: 1.221
SPICE: 0.216

```

► Eval COCO-Stuff 10K

Download the evaluation code from:

- [eval_10k](#) Extract the zip file and put the [eval_10k](#) folder into **Project Root** folder. The folder hierarchy should be:

```

/
├─ data/
├─ model/
├─ utils/
├─ eval_10k/
│   ├── test_stuff_10k/
│   │   └─ *.jpg
│   ├── cocostuff-10k-v1.1.json
│   ├── eval_stuff_10k.py
│   ├── predict_10k.py
│   └─ test_stuff_10k.txt
etc.

```

Files in this zip:

- test_stuff_10k folder: stuff 10k test set (1k images)
- cocostuff-10k-v1.1.json: ground truth file of stuff 10k
- eval_stuff_10k.py: evaluation code for stuff 10k
- predict_10k.py: generate the json file of predictions
- test_stuff_10k.txt: file list of stuff 10k test set

Usage:

1. Use the following command to create prediction file:

```
python eval_10k/predict_10k.py --weights /path/to/weights.pt --source eval_10k/test_stuff_10k --data /path/to/coco.yaml --img 640 --device 0 --export-mask --color-map /path/to/color_map.pickle
```

The prediction file [predictions_semantic.json](#) will be created under the predict result folder (default: [runs/predict-sem/exp{n}/](#))

2. Then, use the following command for evaluation:

```
python eval_stuff_10k.py --gt eval_10k/cocostuff-10k-v1.1.json --pred /path/to/predictions_semantic.json
```

Training

Generalist Visual Language Tasks

Use DDP for training:

```
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 caption/train.py --data coco.yaml --epochs 60 --batch 64 --img 640 --cfg models/caption/gyolo.yaml --name gyolo --weights '' --hyp data/hyps/hyp.scratch-cap.yaml --device 0,1,2,3,4,5,6,7 --optimizer AdamW --flat-cos-lr --no-overlap --close-mosaic 2 --save-period 1 --noval --noplots
```

Cider Optimization for Improving Image Captioning:

```
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 caption/tune.py --weights 'gyolo.pt' --cfg
models/caption/gyolo.yaml --name gyolo-co --data data/coco.yaml --hyp data/hyps/hyp.scratch-cap.yaml --epochs 10 --
batch-size 64 --img 640 --device 0,1,2,3,4,5,6,7 --optimizer AdamW --save-period 1
```

Use torchrun for distributed training on slurm cluster:

```
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=$NODE_RANK --master_addr=$MASTER_ADDR --master_port=9527
caption/train.py --device 0,1,2,3,4,5,6,7 --batch 128 --epochs 40 --workers 16 --data data/coco.yaml --img 640 --cfg
models/caption/gyolo.yaml --name gyolo --weights '' --hyp data/hyps/hyp.scratch-cap.yaml --optimizer AdamW --flat-cos-
lr --no-overlap --close-mosaic 2 --save-period 1 --noplots
```

Cider Optimization for Improving Image Captioning:

```
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=$NODE_RANK --master_addr=$MASTER_ADDR --master_port=9527
caption/tune.py --weights 'gyolo.pt' --cfg models/caption/gyolo.yaml --name gyolo-co --data data/coco.yaml --hyp
data/hyps/hyp.scratch-cap.yaml --epochs 10 --batch-size 128 --img 640 --device 0,1,2,3,4,5,6,7 --optimizer AdamW --
save-period 1
```

Note: Please replace \$MASTER_ADDR and \$NODE_RANK to your own machine.

Inference

```
python caption/predict.py --weights 'gyolo.pt' --source $SOURCE --data data/coco.yaml --img 640 --device 0 --export-
mask
```

Note: Please replace \$SOURCE with the image/video you want to infer.

Citation

```
@inproceedings{wang2024yolov9,
  title={Generalist YOLO}: Towards Real-Time End-to-End Multi-Task Visual Language Models},
  author={Chang, Hung-Shuo and Wang, Chien-Yao and Wang, Richard and Chou, Gene and Liao, Hong-Yuan Mark},
  booktitle={IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)},
  year={2025},
  pages={6217-6227}
}
```

Coming Soon► **Coming soon**

- Visual Grounding
- Whole Body Pose Estimation
- Visual Question Answering
- Video Understanding Tasks
- Specific Vision Tasks

Contributors► **Contributors**

- @ws6125 : Most of generic vision tasks and visual language tasks.
- @110598074 : Most of specific vision tasks.
- @F84064014 : Video panoptic segmentation.
- @k22708038 : Visual grounding.
- @HuChengXue : Whole body pose estimation.
- @waue0920 : Multiple node training.
- ...

Acknowledgements► **Reference Codes**

- <https://github.com/WongKinYiu/yolov9>
- <https://github.com/davidnvq/grit>

- <https://github.com/djiajunustc/TransVG>
- <https://github.com/Robertwyq/PanoOcc>
- <https://github.com/junjiehe96/FastInst>
- <https://github.com/fan23j/yolov7-pose-whole-body>
- ...